

Les Fonctions Stockées MySQL

M203 - Gestion des données

BENTALEB SAAD

ISTA MIRLEFT

22/09/2025

- 1 Définition des Fonctions Stockées
- 2 Syntaxe et Création
- 3 Exemples Pratiques
- 4 Gestion des Fonctions
- 5 Exercice d'Application
- 6 Conclusion

Qu'est-ce qu'une fonction stockée MySQL ?

Définition formelle

Une fonction stockée dans MySQL est un ensemble d'instructions SQL qui :

- Exécutent une tâche/opération spécifique
- Renvoient une **valeur unique**
- Peuvent être appelées dans des instructions SQL

Différences avec les procédures stockées

- Paramètres : uniquement **IN** (implicite)
- Retour : une seule valeur définie dans l'en-tête
- Usage : calcul et renvoi d'une valeur unique

Utilisation recommandée pour les calculs et transformations de données

```
DROP FUNCTION IF EXISTS func_name;  
DELIMITER $$  
CREATE FUNCTION func_name(func_parameter(s))  
RETURNS datatype  
{characteristic}  
BEGIN  
    fun_body;  
END $$  
DELIMITER ;
```

Paramètres principaux

- **func_name** : nom de la fonction
- **func_parameter** : liste des paramètres (IN uniquement)
- **datatype** : type de données de la valeur retournée
- **fun_body** : corps avec au moins une instruction RETURN

Types de caractéristiques

- DETERMINISTIC
- READS SQL DATA
- MODIFIES SQL DATA

Autres caractéristiques

- CONTAINS SQL
- NO SQL

Définitions

DETERMINISTIC	Même résultat pour les mêmes entrées
READS SQL DATA	Lecture seule de la base de données
NO SQL	Aucune instruction SQL (calculs purs)

Exemple sans paramètre

```
DELIMITER $$  
CREATE FUNCTION say_hello1()  
RETURNS VARCHAR(20)  
NO SQL  
BEGIN  
    DECLARE msg VARCHAR(20);  
    SET msg = "Hello world";  
    RETURN msg;  
END $$  
DELIMITER ;  
  
SELECT say_hello1();
```

Appel de fonction

```
SELECT func_name(parameter(s))
```

Exemple avec paramètre - Catégorisation

Spécification

Catégorisation des prix : < 50 (A) • 50-99 (B) • 100 (C)

```
DELIMITER $
CREATE FUNCTION category_price(price DOUBLE)
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
    DECLARE category VARCHAR(20);
    IF price < 50 THEN
        SET category = 'Category A';
    ELSEIF price < 100 THEN
        SET category = 'Category B';
    ELSE
        SET category = 'Category C';
    END IF;
    RETURN category;
END $
```

Utilisation dans les requêtes

```
SELECT ProductID, ProductName, Price,  
       category_price(Price) AS Category  
FROM products;
```

Intégration avec les procédures

```
DELIMITER $$  
CREATE PROCEDURE get_productsDetails()  
BEGIN  
    SELECT ProductID, ProductName, Price,  
           category_price(Price)  
    FROM products  
    ORDER BY Price DESC;  
END $$  
DELIMITER ;  
  
CALL get_productsDetails();
```

Lister les fonctions stockées

Toutes les fonctions du serveur

```
SHOW FUNCTION STATUS;
```

Fonctions d'une base spécifique

```
SHOW FUNCTION STATUS WHERE Db = 'w3schools';  
SHOW FUNCTION STATUS WHERE Db LIKE '%w3%';
```

Utilisation de information_schema

```
SELECT routine_name  
FROM information_schema.routines  
WHERE routine_type = 'FUNCTION'  
AND routine_schema = 'database_name';
```

Syntaxe de suppression

```
DROP FUNCTION [IF EXISTS] func_name;
```

Exemple

```
DROP FUNCTION say_hello1;
```

Bonnes pratiques

- Utiliser **IF EXISTS** pour éviter les erreurs
- Documenter les fonctions complexes
- Tester les fonctions avant déploiement

Énoncé

Créer une fonction stockée **calculate_discount** qui :

- ❶ Prend en paramètre un prix (DECIMAL)
- ❷ Retourne le prix avec remise selon les règles :
 - Prix < 100 : aucune remise
 - 100 ≤ Prix < 500 : remise de 5%
 - 500 ≤ Prix < 1000 : remise de 10%
 - Prix ≥ 1000 : remise de 15%
- ❸ Utiliser cette fonction sur la table **products**

Attendu

- Fonction avec gestion des conditions
- Requête d'utilisation avec SELECT
- Test avec différents prix

Solution - Partie 1

```
DELIMITER $$
CREATE FUNCTION calculate_discount(original_price DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE discounted_price DECIMAL(10,2);

    IF original_price < 100 THEN
        SET discounted_price = original_price;
    ELSEIF original_price < 500 THEN
        SET discounted_price = original_price * 0.95;
    ELSEIF original_price < 1000 THEN
        SET discounted_price = original_price * 0.90;
    ELSE
        SET discounted_price = original_price * 0.85;
    END IF;

    RETURN discounted_price;
END $$
```

Utilisation de la fonction

```
SELECT ProductID ,  
       ProductName ,  
       Price AS Prix_Original ,  
       calculate_discount(Price) AS Prix_Remise ,  
       ROUND((Price - calculate_discount(Price)), 2) AS Remise  
FROM products  
ORDER BY Price DESC;
```

Test avec valeurs spécifiques

```
SELECT calculate_discount(50) AS Test1 ,  
       calculate_discount(250) AS Test2 ,  
       calculate_discount(750) AS Test3 ,  
       calculate_discount(1200) AS Test4;
```

Avantages

- Réutilisabilité du code
- Performance optimisée
- Logique centralisée
- Intégration SQL native

Cas d'usage

- Calculs complexes
- Transformations de données
- Validations métier
- Formatage standardisé

Points clés à retenir

- Une fonction retourne **une seule valeur**
- Paramètres en **entrée uniquement**
- Utilisable dans les requêtes SELECT
- Gérer les caractéristiques selon l'usage